

Способы развёртывания Node.js REST API и развёртывание на PaaS Render

От локального Express-приложения до публичного HTTPS API

Цели лекции

- Понять, что означает deployment серверного приложения.
- Сравнить основные способы размещения Node.js REST API.
- Разобрать модель PaaS и место Render в процессе deployment.
- Подготовить Express + Prisma приложение к запуску в облачной среде.
- Создать Render Postgres и подключить REST API через DATABASE_URL.
- Опубликовать API и проверить health endpoints, logs и auto-deploy.

Связь с предыдущей лекцией

- CI проверяет код, deployment делает проверенную версию приложения доступной пользователям.
- На предыдущей лекции GitHub Actions выполнял `npm ci`, `Prisma` и `tests`.
- Теперь следующий шаг — запустить приложение вне компьютера разработчика.
- Render может автоматически deploy-ить изменения из связанной Git-ветки.
- Render также умеет ждать успешных CI checks перед auto-deploy.



1. Что такое развёртывание серверного приложения

От localhost к постоянно доступному REST API

Development и Production

Development

- Приложение запускается на ПК разработчика.
- Адрес обычно localhost.
- Используются локальные environment variables.
- Сервер может быть выключен в любой момент.
- Ошибки изучаются в локальном terminal.

Production

- Приложение работает на удалённой инфраструктуре.
- API имеет публичный HTTPS URL.
- Secrets хранятся отдельно от исходного кода.
- Процесс должен стабильно запускаться и перезапускаться.
- Logs и health checks используются для диагностики.

Deployment

- Deployment — процесс подготовки и запуска определённой версии приложения в среде, где оно будет обслуживать реальные запросы.
- Получение исходного кода или готового артефакта.
- Установка production dependencies.
- Настройка environment variables и secrets.
- Подготовка базы данных.
- Запуск серверного процесса.
- Проверка доступности и работоспособности.

Разрабатывай
→ Публикуй
→ Делись
→ Развивай!

Путь REST API от разработки до Production

От локального кода — до публичного API, доступного всему миру

Один код.
Реальное влияние
в продакшене!

1

Local development

Разработка на локальном компьютере



ex

Express API

Разрабатываем REST API
на Node.js + Express

>_

localhost

Запускаем и тестируем локально
(например, <http://localhost:3000>)

</>

Редактирование кода

Пишем, изменяем, тестируем

2

Git repository

Хранение кода в Git



Commit

Сохраняем изменения
в историю



Push

Отправляем код
в удалённый репозиторий



Version history

Ведём историю изменений
и можем вернуться к любой версии

3

Deployment platform

Сборка и публикация



Build

Платформа собирает проект
(устанавливает зависимости, запускает сборку)



Environment variables

Настраиваем переменные окружения
(API_KEYS, DATABASE_URL и т.д.)



Deploy

Публикуем приложение на платформе



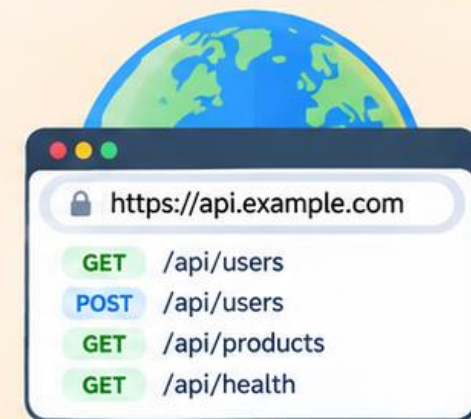
Logs

Следим за логами и состоянием приложения

4

Public API

Публично доступный REST API



HTTPS

Безопасное подключение
через HTTPS



Публичный URL

Например: <https://api.example.com>



REST endpoints

API доступен для любых клиентов
(веб, мобильные приложения и т.д.)



Код разрабатывается локально



сохраняется в Git



публикуется на платформе



становится доступным
как публичный API

ОТ ЛОКАЛЬНОЙ РАЗРАБОТКИ — К РЕАЛЬНЫМ ПОЛЬЗОВАТЕЛЯМ

API
меняет идеи
на возможности

Что должно существовать после deployment

- Запущенный процесс Node.js / Express.
- Стабильное подключение к PostgreSQL.
- Публичный URL для REST API.
- HTTPS.
- Environment variables и secrets.
- Health endpoint.
- Logs запуска и выполнения запросов.

Что переносится в Production, а что нет

Элемент	Переносим	Комментарий
Исходный код	Да	Получаем из Git repository
package.json	Да	Описывает dependencies и scripts
package-lock.json	Да	Фиксирует версии dependencies
.env	Нет	Secrets задаём средствами платформы
node_modules	Нет	Устанавливаем заново
Prisma schema/migrations	Да	Нужны для Client и структуры БД
Локальная PostgreSQL	Нет	Используем отдельную production database

Общий цикл deployment

- 1 Source** Получить выбранную версию проекта из Git.
- 2 Build** Установить dependencies и выполнить подготовительные команды.
- 3 Configure** Передать environment variables и secrets.
- 4 Database** Подключить PostgreSQL и применить migrations.
- 5 Start** Запустить Node.js процесс.
- 6 Verify** Проверить health endpoint, API и logs.

2. Основные способы развёртывания Node.js REST API

VPS, PaaS, containers и serverless

Четыре распространённых подхода

● VPS / Linux server

Самостоятельно устанавливаем Node.js, PostgreSQL, systemd, Nginx и отвечаем за сервер.

● PaaS

Платформа берёт на себя значительную часть инфраструктуры, а разработчик задаёт код и настройки запуска.

● Containers

Приложение упаковывается в container image и запускается в контейнерной инфраструктуре.

● Serverless

Код выполняется платформой по запросу без постоянного управления серверным процессом.

Сравнение способов deployment

Подход	Контроль	Сложность старта	Инфраструктура	Для нашего курса
VPS/Linux	Высокий	Средняя	Настраиваем сами	Подробно в лекции 8
PaaS	Средний	Низкая	Большую часть даёт платформа	Основная тема этой лекции
Containers	Высокий	Средняя/высокая	Container runtime/orchestration	Только обзорно
Serverless	Ниже на уровне сервера	Низкая/средняя	Управляет платформа	Только обзорно

VPS и PaaS

VPS

- Есть доступ к ОС.
- Самостоятельная установка ПО.
- Самостоятельный reverse proxy.
- Больше контроля.
- Больше административной работы.

PaaS

- ОС скрыта от разработчика.
- Build/start задаются командами.
- HTTPS и публичный endpoint предоставляет платформа.
- Быстрее первый deployment.
- Меньше контроля над инфраструктурой.

Где здесь Docker

- Containers — один из способов упаковки и доставки приложения, но не обязательное условие deployment Node.js.
- Node.js можно развернуть напрямую на Linux.
- PaaS может запускать приложение без собственного Dockerfile.
- Некоторые PaaS также поддерживают deployment готового container image.
- В этом курсе контейнеризацию подробно не изучаем.

Что такое PaaS

- Platform as a Service (PaaS) предоставляет готовую среду для сборки, запуска и публикации приложения, скрывая большую часть управления сервером.
- Подключаем Git repository.
- Задаём build command.
- Задаём start command.
- Добавляем environment variables.
- Получаем public URL и HTTPS.
- Просматриваем deploy history и logs.

VPS vs PaaS

Что настраивает разработчик и что берёт на себя платформа

VPS

Вы получаете виртуальный сервер и настраиваете почти всё сами.



Разработчик настраивает и управляет:



OC



Node.js



Nginx



PostgreSQL



Prisma / migrations



systemd / process manager



SSL



deploy



backups



monitoring



security updates



scaling

Больше контроля, но больше ответственности.

Вы сами настраиваете, обновляете и поддерживаете всю инфраструктуру.



Кто за что отвечает

VPS:

Разработчик / администратор — почти всё.

PaaS

Платформа берёт на себя инфраструктуру, а вы фокусируетесь на приложении.



Разработчик в основном занимается:



код



config/env (переменные окружения)



build/start command



migrations



проверка приложения

Платформа берёт на себя:



серверная инфраструктура



runtime



deploy из Git



HTTPS



logs



autoscaling / scaling tools



health (проверки состояния)



restart (автоперезапуск)



managed environment
(управляемое окружение)

Проще и быстрее запустить, меньше администрирования.

Вы получаете готовую платформу и не управляете низкоуровневой инфраструктурой.

PaaS:

Платформа — инфраструктура, разработчик — приложение.



Ключевая идея

VPS: больше контроля, больше ручной настройки.

PaaS: быстрее запуск, меньше администрирования.

Почему для практики выбираем Render

- Поддерживает Node.js web services.
- Можно подключить GitHub repository.
- Предоставляет публичный onrender.com URL.
- Поддерживает managed PostgreSQL.
- Есть environment variables, health checks и logs.
- Поддерживает automatic deploy из связанной Git-ветки.

Путь REST API от разработки до Production

От локальной разработки до публичного API

1. Local development

Разработка и тестирование на своём компьютере



`npm run dev`
`http://localhost:3000`

git push

2. Git repository

Хранение кода и управление версиями



GitHub

- Исходный код
- История изменений
- Ветки и Pull Requests

Автодеплой
(например, через GitHub)

3. Deployment platform

Сборка, настройка и запуск приложения



Другие платформы (PaaS)

Публичный доступ

4. Public API

Ваше API доступно в интернете для пользователей и клиентов



<https://api.yourapp.com>

- ✓ Доступ 24/7
- ✓ HTTPS (безопасное соединение)
- ✓ Готово к использованию в реальных проектах

VPS vs PaaS

Что настраивает разработчик и что берёт на себя платформа



VPS (например, Ubuntu)

Полный контроль, больше ответственности



Устанавливает разработчик



Устанавливает разработчик



Настраивает разработчик



Настраивает разработчик



Настраивает разработчик



Настраивает разработчик



Обновляет разработчик



Настраивает разработчик



PaaS (например, Render)

Простое развёртывание, меньше рутины



Предоставляет платформа



Частично или полностью платформа



Берёт на себя платформа



Автоматически (SSL от платформы)



Встроенные инструменты



Простое масштабирование (1 клик)



Автоматические обновления



Автоматические бэкапы (в плане)

Вывод: VPS — больше гибкости, но больше рутины.
PaaS — быстрее старт, меньше настроек, удобно для учебных и небольших проектов.

Internet Shop на Render

Архитектура приложения в облаке

Client

Браузер / Frontend



- Просмотр товаров
- Оформление заказа
- Работа с аккаунтом

HTTPS
(порт 443)

Web Service
(Node.js + Express)



- REST API
- Бизнес-логика
- Обработка запросов

Prisma



- ORM
- Работа с БД
- Миграции
- Типизация

Render PostgreSQL



- База данных магазина
- Товары, заказы, пользователи
- Автоматические бэкапы

Вся инфраструктура на Render

Без настройки серверов, с автоматическим деплоем



Результат:

полноценный интернет-магазин,
доступный в интернете по HTTPS



<https://your-shop.onrender.com>



3. Подготовка Express REST API к Render

PORT, scripts, Prisma и Production configuration

Требование к HTTP-серверу Render

- Web Service должен принимать внешние HTTP-запросы.
- Приложение должно слушать порт из environment variable PORT.
- Сервер должен bind-иться на 0.0.0.0, а не только на localhost.
- Не следует жёстко фиксировать production port в исходном коде.

server.js для Render

JAVASCRIPT

```
import app from './app.js';

const PORT = Number(process.env.PORT ?? 3000);
const HOST = '0.0.0.0';

const server = app.listen(PORT, HOST, () => {
  console.log(`API started on ${HOST}:${PORT}`);
});
```

Что важно

- PORT берётся из environment Render.
- 3000 остаётся удобным fallback для локальной разработки.
- 0.0.0.0 позволяет платформе направлять входящие запросы в Express.
- server сохраняем для graceful shutdown.

Неправильно и правильно

Неправильно для cloud

- `app.listen(3000, '127.0.0.1')`
- Порт зафиксирован в коде.
- Сервер слушает только loopback interface.
- Платформа может не обнаружить доступный HTTP port.

Правильно

- `process.env.PORT`
- Fallback только для local development.
- Host 0.0.0.0.
- Порт задаётся инфраструктурой.

Production scripts в package.json

JSON

```
{
  "scripts": {
    "start": "node server.js",
    "test": "node --experimental-vm-modules node_modules/jest/bin/jest.js --runInBand",
    "prisma:generate": "prisma generate",
    "db:deploy": "prisma migrate deploy"
  }
}
```

Что важно

- start — команда постоянного запуска REST API.
- test остаётся для CI и локальной проверки.
- prisma:generate подготавливает Prisma Client.
- db:deploy применяет уже созданные production migrations.

Node.js version

- Production и development должны использовать совместимые версии Node.js.
- В нашем учебном проекте остаёмся на Node.js 22.
- Версию желательно фиксировать в настройках проекта или платформы.
- Это уменьшает риск различий между локальной средой и deployment.

Фиксация Node.js 22 через package.json

JSON

```
{  
  "engines": {  
    "node": "22.x"  
  }  
}
```

Production environment variables

- `NODE_ENV=production`.
- `DATABASE_URL` — connection string PostgreSQL.
- `JWT_SECRET` — секрет подписи JWT.
- `CORS_ORIGIN` — разрешённый frontend origin, если требуется.
- Другие API keys и secrets задаются в Render Dashboard.
- `.env` не загружаем в Git repository.

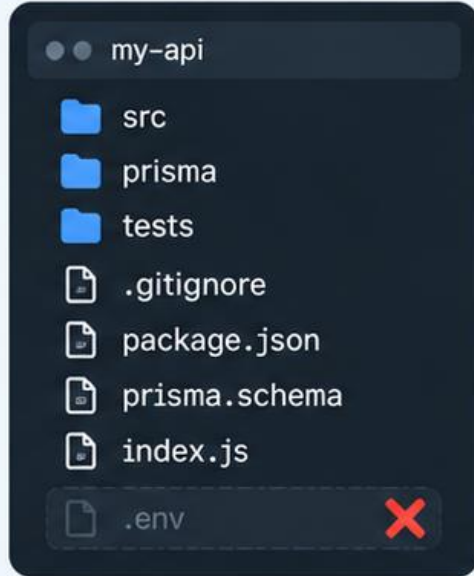
Конфигурация без .env

Переменные окружения задаются в Render, а не в файле .env



Repository (GitHub)

Код приложения без .env



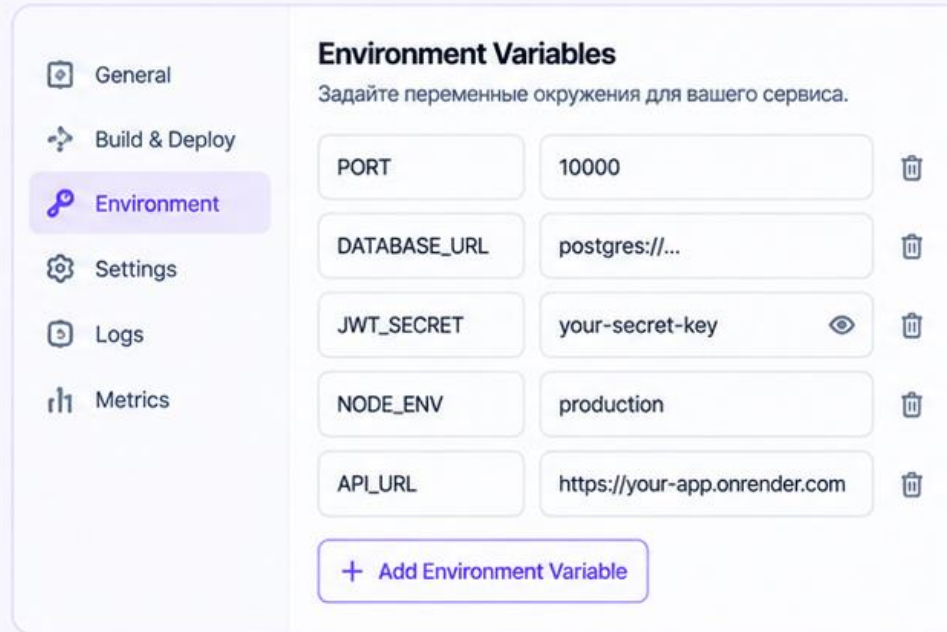
git push

Код
отправляется
в GitHub

2

Render Environment

Переменные окружения задаются
в настройках сервиса на Render



Переменные хранятся в Render

Они подставляются в окружение приложения
при запуске сервиса.

Deploy

Render
запускает
приложение
с указанными
переменными

3

Node.js process

Приложение получает значения
через process.env



Приложение работает с переменными,
которые предоставляет Render,
без использования файла .env.



Результат: чувствительные данные хранятся в Render, а не в репозитории. Приложение автоматически получает их при запуске.

Проверка проекта перед deployment



Проект находится в Git repository.



package-lock.json актуален.



npm test проходит.



npm start запускает API локально.



server.js использует process.env.PORT.



Prisma migrations находятся в prisma/migrations.



.env добавлен в .gitignore.



/health/live и /health/ready работают.

Что будет делать Render

- Получит код из связанной Git-ветки.
- Запустит Build Command.
- Создаст новую версию сервиса.
- Передаст environment variables.
- Запустит Start Command.
- Проверит доступность web service.
- После успешного deploy предоставит публичный HTTPS URL.



4. PostgreSQL и Prisma на Render

Managed database, DATABASE_URL и migrations

Render Postgres

- Render Postgres — управляемая PostgreSQL database, которую можно использовать отдельно от Web Service.
- База создаётся как отдельный ресурс.
- Имеет собственные credentials и connection URLs.
- Web Service подключается через DATABASE_URL.
- Для production не используем локальный localhost.

Internal URL и External URL

Internal Database URL

- Для Render services в том же регионе.
- Трафик идёт по private network.
- Обычно ниже latency.
- Предпочтительный вариант для Web Service.

External Database URL

- Для подключения извне Render.
- Например, локальный database client.
- Трафик идёт через public network.
- Используется только когда internal connection недоступен.



Внутренняя сеть Render

- Только между сервисами Render
- Без публичного доступа
- Быстро и безопасно

Web Service

(Node.js + Express)



- ✓ Ваш REST API
- ✓ Подключается к базе по внутреннему URL
- ✓ Не требует публичного доступа к базе данных

Подключение по внутренней сети

Render private network

Внутренняя сеть (private network)

Internal Database URL

```
postgresql://user:password@  
<internal-host>:5432/database
```

- ✓ Используется внутри Render
- ✓ Недоступен из интернета
- ✓ Автоматически предоставляется после создания PostgreSQL

Внутреннее соединение (порт 5432)

PostgreSQL

(Managed Database)



- ✓ Управляемая база данных на Render
- ✓ Доступна только во внутренней сети
- ✓ Автоматические бэкапы
- ✓ Высокая доступность

Пример в коде (Prisma)

```
// schema.prisma  
datasource db {  
  provider = "postgresql"  
  url      = env("DATABASE_URL") // Internal URL из Render  
}
```

Где взять Internal Database URL?

- 1 Откройте ваш PostgreSQL сервис на Render
- 2 Перейдите во вкладку "Connection"
- 3 Скопируйте значение "Internal Database URL"
- 4 Добавьте его в переменную DATABASE_URL вашего Web Service

Пример переменной окружения на Render

```
DATABASE_URL=  
postgresql://user:password@dpg-abc123:5432/mydb
```



Не используйте External Database URL в Web Service, если оба сервиса находятся на Render. Внутренний URL быстрее, безопаснее и не требует публичного доступа.



Результат: Web Service и PostgreSQL безопасно обмениваются данными через внутреннюю сеть Render.



render Build. Deploy. Scale.

Почему сервис и БД размещаем в одном регионе

- Можно использовать Internal Database URL.
- Запросы к базе не выходят в публичную сеть.
- Уменьшается network latency.
- Архитектура становится проще для учебного проекта.

Prisma migrations в Production

- `prisma migrate deploy` применяет уже созданные migrations и предназначен для staging/production сценариев.
- Не создаёт новые migrations.
- Не сбрасывает production database.
- Не заменяет `prisma generate`.
- Должен выполняться до работы версии приложения, которая ожидает новую схему.

Где запускать migrations на Render

Учебный простой вариант

- Start Command: `npm run db:deploy && npm start`.
- Подходит для демонстрационного проекта.
- `migrate deploy` безопасно повторно проверяет pending migrations.
- Не требует отдельного pre-deploy шага.

Production-подход платформы

- Pre-Deploy Command: `npx prisma migrate deploy`.
- Выполняется до запуска новой версии.
- Чище разделяет migration и runtime.
- Доступность функции зависит от плана Render.

5. Развёртывание Internet Shop API на Render

Git repository → PostgreSQL → Web Service → public URL

Шаг 1. Подготовить Git repository

- 1 Проверить branch** Используем main как deployment branch.
- 2 Проверить lockfile** package-lock.json должен быть в repository.
- 3 Проверить migrations** Каталог prisma/migrations должен быть закоммичен.
- 4 Проверить secrets** .env не должен находиться в repository.
- 5 Push** Отправить актуальную версию проекта в GitHub.

Шаг 2. Создать Render Postgres

- 1 New PostgreSQL** Создать новую PostgreSQL database в Render Dashboard.
- 2 Region** Выбрать регион, который затем используем и для Web Service.
- 3 Wait** Дождаться создания database.
- 4 Connect** Открыть параметры подключения.
- 5 Internal URL** Скопировать Internal Database URL для Web Service.

Шаг 3. Создать Web Service

- 1 New Web Service** Создать публичный web service.
- 2 Connect Git** Подключить GitHub account и выбрать repository.
- 3 Branch** Выбрать main.
- 4 Language** Выбрать Node.
- 5 Region** Выбрать тот же регион, что у PostgreSQL.

Основные настройки Web Service

Поле Render	Значение для Internet Shop
Language	Node
Branch	main
Build Command	npm ci && npx prisma generate
Start Command	npm run db:deploy && npm start
NODE_ENV	production
DATABASE_URL	Internal Database URL Render Postgres
JWT_SECRET	Отдельный production secret
Health Check Path	/health/ready

Build Command

BASH

```
npm ci && npx prisma generate
```

Start Command для учебного deployment

BASH

```
npm run db:deploy && npm start
```

Что важно

- Сначала prisma migrate deploy проверяет и применяет pending migrations.
- Если migration завершилась ошибкой, Node.js process не стартует.
- После успешной migration запускается npm start.
- Для более серьёзной production-среды migration лучше вынести в отдельный pre-deploy этап.

Шаг 4. Добавить Environment Variables

- 1 DATABASE_URL** Вставить Internal Database URL созданной PostgreSQL.
- 2 NODE_ENV** Задать production.
- 3 JWT_SECRET** Создать отдельное случайное production значение.
- 4 CORS_ORIGIN** Добавить только если API ограничивает origin frontend.
- 5 Save** Сохранить variables и запустить deploy.

Ваш
Node.js API
в продакшене
на Render

Render deploy pipeline



От кода до работающего API на Render



Код из репозитория проходит сборку, получает переменные окружения, применяет миграции, запускает приложение, проходит health check и становится доступным по публичному URL.

Публичный URL REST API

- После успешного deploy Web Service получает onrender.com URL.
- HTTPS предоставляется платформой.
- К URL добавляем привычные маршруты API.
- Например: /api/products, /api/auth/login, /health/live.
- Frontend или Postman теперь обращаются не к localhost, а к public URL.

6. Проверка, logs и автоматический redeploy

Как убедиться, что опубликованный REST API действительно работает

Smoke test после первого deployment

- 1 Liveness** Открыть `/health/live` и получить успешный ответ.
- 2 Readiness** Открыть `/health/ready` и проверить PostgreSQL connection.
- 3 GET** Проверить публичный GET `/api/products`.
- 4 Authentication** Проверить login и получение JWT.
- 5 Protected endpoint** Отправить запрос с Authorization header.

HTTP Health Check на Render

- Render может отправлять GET-запрос на выбранный health path.
- Успешным считается ответ 2xx или 3xx.
- Для Internet Shop используем /health/ready.
- Readiness может выполнить простую query к PostgreSQL.
- Health check помогает платформе определить готовность новой версии.

Readiness endpoint

JAVASCRIPT

```
app.get('/health/ready', async (req, res) => {  
  try {  
    await prisma.$queryRaw`SELECT 1`;  
    res.status(200).json({ status: 'ready' });  
  } catch (error) {  
    res.status(503).json({ status: 'not_ready' });  
  }  
});
```

Что важно

- Endpoint проверяет не только Express, но и доступность database.
- 200 означает, что приложение готово обслуживать запросы.
- 503 означает, что зависимость недоступна.
- Endpoint не должен раскрывать credentials или stack trace.

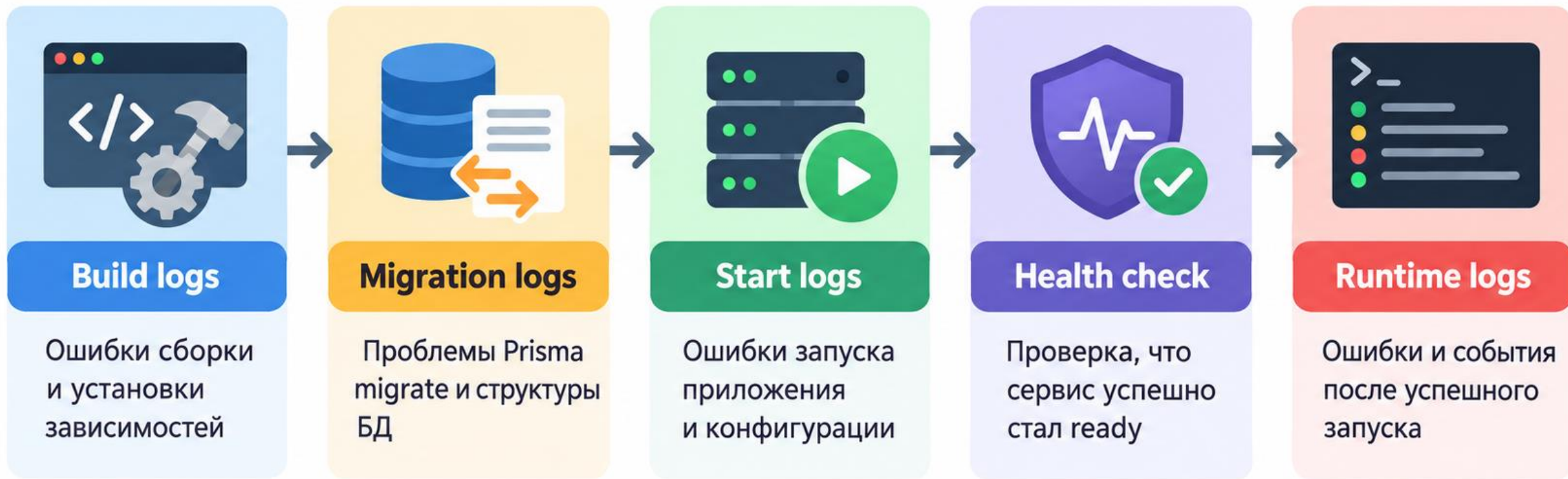
Logs в Render

- Build logs показывают установку dependencies и prisma generate.
- Deploy/start logs показывают migrations и запуск Node.js.
- Runtime logs содержат console/Pino output приложения.
- При ошибке сначала определяем этап: build, migration, start или runtime.
- Logs позволяют диагностировать failure без доступа к ОС сервера.

Типичные ошибки первого deployment

Симптом	Причина	Что проверить
No open ports detected	Приложение не слушает ожидаемый port	process.env.PORT и host 0.0.0.0
Prisma connection error	Неверный DATABASE_URL	Internal URL, region, variable name
Migration failed	Проблема migration files	prisma/migrations и schema
Application exited	Ошибка Start Command или startup validation	Deploy logs и environment variables
401/403	API работает, но запрос не авторизован	JWT и Authorization header
CORS error в браузере	Неверный allowed origin	CORS_ORIGIN и middleware

Диагностика **Render**



Идея: при проблемах деплоя анализируйте логи по этапам — от сборки к работе приложения.

Auto-Deploy после push

- Render связывает Web Service с конкретной Git-веткой.
- По умолчанию изменения в связанной ветке могут автоматически запускать новый deploy.
- Можно настроить deployment только после успешных CI checks.
- Это связывает прошлую лекцию о GitHub Actions с текущим deployment.

Render deploy pipeline

От коммита в GitHub до работающего API



render

Build. Deploy. Scale.



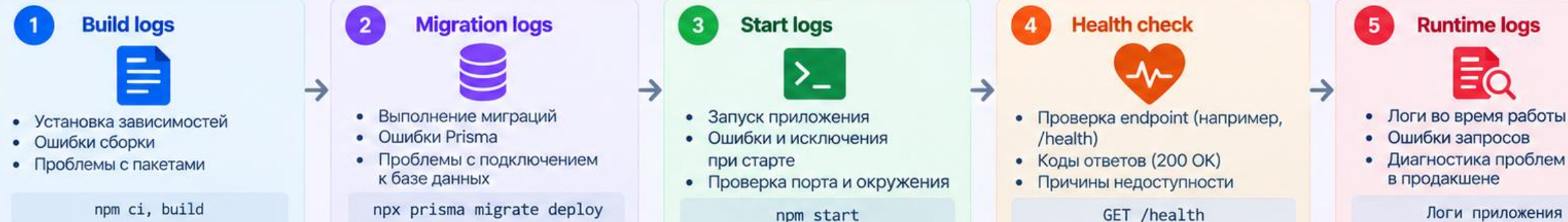
Диагностика Render

Где смотреть, если что-то пошло не так?



render

Build. Deploy. Scale.



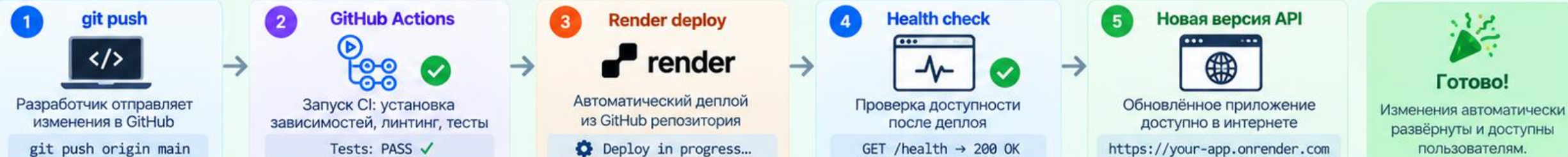
CI + Render

Автоматическое тестирование и деплой при каждом пуше



render

Build. Deploy. Scale.



Практическая задача

Опубликовать Internet Shop REST API на Render и получить рабочий публичный HTTPS endpoint.

Задание

- Создать Render Postgres.
- Создать Web Service из GitHub repository.
- Настроить Build Command и Start Command.
- Передать DATABASE_URL, NODE_ENV и JWT_SECRET.
- Включить health check /health/ready.
- Проверить GET endpoint и authentication.
- Найти deployment в Deploys и открыть logs.

Критерии проверки

- REST API доступен по публичному HTTPS URL.
- Prisma подключается к Render Postgres.
- Migrations применены.
- Health check проходит.
- После нового push выполняется новый deployment.

Проверка понимания

- 1 Чем PaaS отличается от VPS с точки зрения управления инфраструктурой?
- 2 Почему Express на Render должен использовать `process.env.PORT`?
- 3 Какой Render Postgres URL предпочтителен для Web Service в том же регионе?
- 4 Для чего используется `prisma migrate deploy`?
- 5 Почему `.env` нельзя хранить в Git repository?
- 6 Где искать причину, если deployment завершился failure?

Итоги лекции

1

Node.js REST API можно развёртывать на VPS, PaaS, container infrastructure и serverless platforms.

2

PaaS уменьшает объём ручного администрирования инфраструктуры.

3

Render Web Service получает код из Git repository и выполняет Build/Start commands.

4

Express должен слушать process.env.PORT на 0.0.0.0.

5

Render Postgres подключается через DATABASE_URL; внутри Render предпочтителен Internal Database URL.

6

Prisma Client генерируется при build, migrations применяются перед запуском новой версии.

7

Environment variables и secrets хранятся вне repository.

8

Health checks и logs позволяют проверить и диагностировать deployment.

9

Следующая лекция: развёртывание того же REST API вручную на Linux-сервере.